

DBMS PRACTICAL SETS

PS-1(1) & PS-4(2)

■ Student Course Management System (SQL Solution)

◆ 1. Create Student Table

```
CREATE TABLE Student (  
  rollno INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  address VARCHAR(100),  
  phno VARCHAR(15),  
  email_id VARCHAR(50) UNIQUE,  
  marks INT CHECK (marks BETWEEN 0 AND 100)  
);
```

◆ 2. Create Courses Table (with FOREIGN KEY + CASCADE)

```
CREATE TABLE Courses (  
  c_id INT PRIMARY KEY,  
  cname VARCHAR(50),  
  rollno INT,  
  FOREIGN KEY (rollno) REFERENCES Student(rollno)  
  ON DELETE CASCADE  
);
```

◆ 3. ALTER Table (Example)

👉 Add a new column:

```
ALTER TABLE Student ADD age INT;
```

◆ 4. RENAME Table

```
ALTER TABLE Courses RENAME TO Course_Details;
```

◆ 5. INSERT Records (7–10 entries)

Student Table

```
INSERT INTO Student VALUES (1, 'Amit', 'Pune', '9876543210', 'amit@gmail.com', 85, 20);  
INSERT INTO Student VALUES (2, 'Riya', 'Mumbai', '9876543211', 'riya@gmail.com', 92, 21);  
INSERT INTO Student VALUES (3, 'Karan', 'Delhi', '9876543212', 'karan@gmail.com', 76, 22);  
INSERT INTO Student VALUES (4, 'Neha', 'Pune', '9876543213', 'neha@gmail.com', 65, 20);  
INSERT INTO Student VALUES (5, 'Rahul', 'Nashik', '9876543214', 'rahul@gmail.com', 55, 23);  
INSERT INTO Student VALUES (6, 'Sneha', 'Nagpur', '9876543215', 'sneha@gmail.com', 88, 21);  
INSERT INTO Student VALUES (7, 'Arjun', 'Pune', '9876543216', 'arjun@gmail.com', 90, 22);
```

Course_Details Table

```
INSERT INTO Course_Details VALUES (101, 'DBMS', 1);  
INSERT INTO Course_Details VALUES (102, 'OS', 2);  
INSERT INTO Course_Details VALUES (103, 'CN', 3);  
INSERT INTO Course_Details VALUES (104, 'AI', 4);  
INSERT INTO Course_Details VALUES (105, 'ML', 5);  
INSERT INTO Course_Details VALUES (106, 'Java', 6);  
INSERT INTO Course_Details VALUES (107, 'Python', 7);
```

◆ 6. Constraint Enforcement (Invalid Inserts)

✗ Marks > 100 (CHECK constraint)

```
INSERT INTO Student VALUES (8, 'Test', 'Pune', '9999999999', 'test@gmail.com', 150, 20);
```

👉 ✗ This will FAIL

✗ Duplicate Email (UNIQUE constraint)

```
INSERT INTO Student VALUES (9, 'Test2', 'Pune', '9999999998', 'amit@gmail.com', 80, 20);
```

👉 ✗ This will FAIL

✗ NULL Name (NOT NULL constraint)

```
INSERT INTO Student VALUES (10, NULL, 'Pune', '9999999997', 'test2@gmail.com', 70, 20);
```

👉 ✗ This will FAIL

◆ 7. Demonstrate ON DELETE CASCADE

```
DELETE FROM Student WHERE rollno = 1;
```

- Delete student with rollno = 1
- Automatically delete related record from Course_Details

◆ 8. TRUNCATE Table

```
TRUNCATE TABLE Course_Details;
```

◆ 9. DROP Table

```
DROP TABLE Course_Details;
```

🎯 Final Viva Points (VERY IMPORTANT)

- PRIMARY KEY → uniquely identifies record
- FOREIGN KEY → maintains relationship
- ON DELETE CASCADE → deletes child records automatically
- CHECK → restricts value range
- UNIQUE → no duplicate values
- NOT NULL → field cannot be empty

PS-1(2) & PS-2(2)

MongoDB: Games Collection Solution

◆ 1. Create Collection

use gameDB

```
db.createCollection("games")
```

◆ 2. Insert 5 Documents

```
db.games.insertMany([
  { name: "BGMI", gametype: "Battle Royale", rating: 85 },
  { name: "Valorant", gametype: "FPS", rating: 90 },
  { name: "FIFA 23", gametype: "Sports", rating: 88 },
  { name: "Minecraft", gametype: "Sandbox", rating: 95 },
  { name: "Call of Duty", gametype: "FPS", rating: 92 }
])
```

◆ 3. Display All Games

```
db.games.find()
```

◆ 4. Top 3 Highest Rated Games

```
db.games.find().sort({ rating: -1 }).limit(3)
```

👉 -1 → descending order (highest first)

◆ 5. Update Two Favourite Games (Add Achievements)

👉 Example: Minecraft & Valorant

```
db.games.updateOne(
  { name: "Minecraft" },
  { $set: { achievements: ["Game Master", "Speed Demon"] } }
)
```

```
db.games.updateOne(
  { name: "Valorant" },
  { $set: { achievements: ["Game Master", "Speed Demon"] } }
)
```

◆ 6. Games with BOTH Achievements

```
db.games.find({
  achievements: { $all: ["Game Master", "Speed Demon"] }
})
```

👉 \$all ensures both values exist

◆ 7. Games Having Achievements (only those)

```
db.games.find({
  achievements: { $exists: true }
})
```

👉 Returns only documents where achievements field exists

Viva Points (Important 🔥)

- insertMany() → inserts multiple documents
- find() → retrieve data
- sort({ field: -1 }) → descending order
- limit() → restrict number of results
- \$set → update/add field
- \$all → match all elements in array
- \$exists → check field existence

📄 Employees & Departments – Full SQL Solution

◆ 1. Create Tables (with keys)

```
CREATE TABLE Departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);
CREATE TABLE Employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  salary INT,
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);
```

◆ 2. Insert Records

Departments (3–5 records)

```
INSERT INTO Departments VALUES (1, 'HR');
INSERT INTO Departments VALUES (2, 'IT');
INSERT INTO Departments VALUES (3, 'Finance');
INSERT INTO Departments VALUES (4, 'Marketing');
```

Employees (8–10 records)

```
INSERT INTO Employees VALUES (101, 'Amit', 50000, 1);
INSERT INTO Employees VALUES (102, 'Riya', 65000, 2);
INSERT INTO Employees VALUES (103, 'Karan', 70000, 2);
INSERT INTO Employees VALUES (104, 'Neha', 45000, 3);
INSERT INTO Employees VALUES (105, 'Rahul', 80000, 2);
INSERT INTO Employees VALUES (106, 'Sneha', 30000, 4);
INSERT INTO Employees VALUES (107, 'Arjun', 90000, 1);
INSERT INTO Employees VALUES (108, 'Priya', 60000, NULL);
INSERT INTO Employees VALUES (109, 'Vikas', 75000, 3);
```

◆ 3. Employees earning more than average salary

```
SELECT *
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

◆ 4. Employee(s) with highest salary

```
SELECT *
FROM Employees
WHERE salary = (SELECT MAX(salary) FROM Employees);
```

◆ 5. Salary > average of their department (Correlated Subquery)

```
SELECT e1.*
FROM Employees e1
WHERE salary > (
  SELECT AVG(salary)
  FROM Employees e2
  WHERE e1.dept_id = e2.dept_id
);
```

👉 This is **very important** — examiner loves this.

◆ 6. Employees in departments where max salary > 60000

```
SELECT *
FROM Employees
WHERE dept_id IN (
  SELECT dept_id
  FROM Employees
  GROUP BY dept_id
  HAVING MAX(salary) > 60000
);
```

◆ 7. Employees NOT assigned to any department (using NOT IN)

```
SELECT *
FROM Employees
WHERE dept_id NOT IN (SELECT dept_id FROM Departments);
```

👉 Also works if dept_id IS NULL, but this is what examiner expects.

◆ 8. Using EXISTS (important concept)

👉 Since your question says “*specific condition*”, we assume example condition like dept_name = ‘IT’

```
SELECT *
FROM Employees e
WHERE EXISTS (
  SELECT *
  FROM Departments d
  WHERE e.dept_id = d.dept_id
  AND d.dept_name = 'IT'
);
```

Banking Management System (SQL Solution)

◆ 1. Create Tables

Customers Table

```
CREATE TABLE Customers (
  cust_id INT PRIMARY KEY,
  name VARCHAR(50),
  balance INT,
  city VARCHAR(50)
```

```
);
```

Transactions Table

```
CREATE TABLE Transactions (
  txn_id INT PRIMARY KEY,
  cust_id INT,
  amount INT,
  txn_type VARCHAR(20),
  FOREIGN KEY (cust_id) REFERENCES Customers(cust_id)
```

```
);
```

◆ 2. Insert Records

Customers

```
INSERT INTO Customers VALUES (1, 'Amit', 5000, 'Pune');
INSERT INTO Customers VALUES (2, 'Riya', 8000, 'Mumbai');
INSERT INTO Customers VALUES (3, 'Karan', 12000, 'Delhi');
INSERT INTO Customers VALUES (4, 'Neha', 7000, 'Pune');
INSERT INTO Customers VALUES (5, 'Rahul', 15000, 'Nagpur');
```

Transactions

```
INSERT INTO Transactions VALUES (101, 1, 2000, 'Deposit');
INSERT INTO Transactions VALUES (102, 2, 1000, 'Withdraw');
INSERT INTO Transactions VALUES (103, 3, 3000, 'Deposit');
INSERT INTO Transactions VALUES (104, 4, 1500, 'Withdraw');
INSERT INTO Transactions VALUES (105, 5, 5000, 'Deposit');
```

◆ 3. Update Balance (Deposit / Withdrawal)

Deposit → balance + amount

```
UPDATE Customers
SET balance = balance + 2000
WHERE cust_id = 1;
```

Withdrawal → balance - amount

```
UPDATE Customers
SET balance = balance - 1000
WHERE cust_id = 2;
```

◆ 4. Delete Inactive Customers

👉 Example: balance < 2000

```
DELETE FROM Customers
WHERE balance < 2000;
```

◆ 5. Arithmetic Operation

```
SELECT name, balance, balance + 1000 AS updated_balance
FROM Customers;
```

◆ 6. Logical Operators (AND / OR)

```
SELECT *
FROM Customers
WHERE city = 'Pune' AND balance > 5000;
SELECT *
FROM Customers
WHERE city = 'Pune' OR city = 'Mumbai';
```

◆ 7. Pattern Matching (LIKE)

```
SELECT *
FROM Customers
WHERE name LIKE 'A%';
```

👉 Names starting with 'A'

◆ 8. String Functions

```
SELECT UPPER(name), LENGTH(name)
FROM Customers;
```

◆ 9. Set Operator (UNION)

```
SELECT name FROM Customers WHERE city = 'Pune'
UNION
SELECT name FROM Customers WHERE city = 'Mumbai';
```

Viva Points (VERY IMPORTANT)

- **Arithmetic** → balance + amount
- **Logical operators** → AND, OR
- **LIKE** → pattern matching (A%)
- **UPPER()** → converts to uppercase
- **LENGTH()** → gives string length **UNION** → combines results, removes duplicates **FOREIGN KEY** → links transactions to customers

PS-6(1) & PS-7(2)

MongoDB Orders Collection (Full Solution)

◆ 1. Create Database & Collection

```
use orderDB
```

```
db.createCollection("orders")
```

◆ 2. Insert Sample Documents (5–7 records)

```
db.orders.insertMany([
  { Id: 1, name: "Amit", product: "Laptop", category: "Electronics", amt: 50000, date: "2024-01-10" },
  { Id: 2, name: "Riya", product: "Mobile", category: "Electronics", amt: 20000, date: "2024-01-12" },
  { Id: 3, name: "Karan", product: "Shoes", category: "Fashion", amt: 3000, date: "2024-01-15" },
  { Id: 4, name: "Neha", product: "Watch", category: "Fashion", amt: 7000, date: "2024-01-18" },
  { Id: 5, name: "Rahul", product: "TV", category: "Electronics", amt: 40000, date: "2024-01-20" },
  { Id: 6, name: "Sneha", product: "Bag", category: "Fashion", amt: 2500, date: "2024-01-22" }
])
```

◆ 3. Display All Documents

```
db.orders.find()
```

Aggregation Queries

◆ 4. Total Sales Amount for Each Category

```
db.orders.aggregate([
  {
    $group: {
      _id: "$category",
      total_sales: { $sum: "$amt" }
    }
  }
])
```

◆ 5. Average Order Amount per Category

```
db.orders.aggregate([
  {
    $group: {
      _id: "$category",
      avg_sales: { $avg: "$amt" }
    }
  }
])
```

◆ 6. Sort Categories by Total Sales (Descending)

```
db.orders.aggregate([
  {
    $group: {
      _id: "$category",
      total_sales: { $sum: "$amt" }
    }
  },
  {
    $sort: { total_sales: -1 }
  }
])
```

Expected Output (Example)

```
{ category: "Electronics", total_sales: 110000 }
{ category: "Fashion", total_sales: 12500 }
```

🔥 Viva Points

- \$group → like SQL GROUP BY
- \$sum → total calculation
- \$avg → average
- \$sort: -1 → descending
- _id → grouping field in MongoDB

SQL Solution – Indexing for Query Performance

1. Create Table

```
CREATE TABLE Users (
  Id INT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(50),
  age INT,
  city VARCHAR(50)
);
```

2. Insert Sample Records

```
INSERT INTO Users VALUES (1, 'Amit', 'amit@gmail.com', 22, 'Mumbai');
INSERT INTO Users VALUES (2, 'Riya', 'riya@gmail.com', 19, 'Pune');
INSERT INTO Users VALUES (3, 'Karan', 'karan@gmail.com', 25, 'Mumbai');
INSERT INTO Users VALUES (4, 'Neha', 'neha@gmail.com', 30, 'Delhi');
INSERT INTO Users VALUES (5, 'Rahul', 'rahul@gmail.com', 27, 'Mumbai');
```

Indexing Queries

3. Create Index on Email Field

```
CREATE INDEX idx_email
ON Users(email);
```

👉 Improves search performance on email

4. Create Compound Index on City and Age

```
CREATE INDEX idx_city_age
ON Users(city, age);
```

👉 Helps when queries use both city and age

5. Query Users from Mumbai aged above 20

```
SELECT *
FROM Users
WHERE city = 'Mumbai'
AND age > 20;
```

Viva Points (VERY IMPORTANT)

What is Index?

- Improves query searching speed
- Similar to book index

Simple Index

```
CREATE INDEX idx_email ON Users(email);
```

👉 Index on one column

Compound Index

```
CREATE INDEX idx_city_age ON Users(city, age);
```

👉 Index on multiple columns

Advantage of Index

- Faster SELECT queries
- Better performance for large data

Disadvantage

- Takes extra storage
- INSERT/UPDATE slightly slower

 SQL Queries for Data Manipulation and Operations◆ **1. Create Table**

```
CREATE TABLE Employee (  
  emp_id INT PRIMARY KEY,  
  emp_name VARCHAR(50),  
  department VARCHAR(50),  
  salary INT,  
  city VARCHAR(50)  
);
```

◆ **2. INSERT Records (DML)**

```
INSERT INTO Employee VALUES (101, 'Amit', 'IT', 50000, 'Pune');  
INSERT INTO Employee VALUES (102, 'Riya', 'HR', 40000, 'Mumbai');  
INSERT INTO Employee VALUES (103, 'Karan', 'Finance', 60000, 'Delhi');  
INSERT INTO Employee VALUES (104, 'Neha', 'IT', 45000, 'Pune');  
INSERT INTO Employee VALUES (105, 'Rahul', 'HR', 35000, 'Nagpur');
```

◆ **3. UPDATE Records**

```
UPDATE Employee  
SET salary = salary + 5000  
WHERE emp_id = 101;
```

👉 Increases Amit's salary by 5000

◆ **4. DELETE Records**

```
DELETE FROM Employee  
WHERE emp_id = 105;
```

👉 Deletes Rahul's record

◆ **5. Arithmetic Operator**

```
SELECT emp_name, salary, salary + 2000 AS updated_salary  
FROM Employee;
```

◆ **6. Logical Operators (AND / OR)**

AND

```
SELECT *  
FROM Employee  
WHERE department = 'IT'  
AND salary > 40000;
```

OR

```
SELECT *  
FROM Employee  
WHERE city = 'Pune'  
OR city = 'Mumbai';
```

◆ **7. Pattern Matching (LIKE)**

```
SELECT *  
FROM Employee  
WHERE emp_name LIKE 'A%';
```

👉 Names starting with A

◆ **8. String Functions**

```
SELECT UPPER(emp_name), LENGTH(emp_name)  
FROM Employee;
```

👉 UPPER() → uppercase

👉 LENGTH() → string length

◆ **9. Set Operator (UNION)**

```
SELECT emp_name FROM Employee WHERE city = 'Pune'
```

UNION

```
SELECT emp_name FROM Employee WHERE city = 'Mumbai';
```

👉 Combines results and removes duplicates

Library Management System (SQL Solution)

◆ 1. Create Tables

Members Table

```
CREATE TABLE Members (
  member_id INT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(50)
);
```

Books Table

```
CREATE TABLE Books (
  book_id INT PRIMARY KEY,
  title VARCHAR(100),
  author VARCHAR(50),
  price INT
);
```

Borrow Table

```
CREATE TABLE Borrow (
  borrow_id INT PRIMARY KEY,
  member_id INT,
  book_id INT,
  borrow_date DATE,
  FOREIGN KEY (member_id) REFERENCES Members(member_id),
  FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

◆ 2. Insert Records

Members

```
INSERT INTO Members VALUES (1, 'Amit', 'amit@gmail.com');
INSERT INTO Members VALUES (2, 'Riya', 'riya@gmail.com');
INSERT INTO Members VALUES (3, 'Karan', 'karan@gmail.com');
```

Books

```
INSERT INTO Books VALUES (101, 'SQL Basics', 'XYZ', 600);
INSERT INTO Books VALUES (102, 'Java Programming', 'ABC', 450);
INSERT INTO Books VALUES (103, 'Advanced SQL', 'XYZ', 800);
INSERT INTO Books VALUES (104, 'Python Guide', 'PQR', 550);
```

Borrow

```
INSERT INTO Borrow VALUES (1, 1, 101, '2024-01-10');
INSERT INTO Borrow VALUES (2, 2, 103, '2024-01-12');
INSERT INTO Borrow VALUES (3, 3, 104, '2024-01-15');
```

◆ 3. UPDATE Records

Update Book Price

```
UPDATE Books
SET price = 700
WHERE book_id = 101;
```

Update Member Email

```
UPDATE Members
SET email = 'riya_new@gmail.com'
WHERE member_id = 2;
```

◆ 4. DELETE Records

👉 Delete returned/lost book record

```
DELETE FROM Borrow
WHERE borrow_id = 3;
```

◆ 5. Arithmetic Operation (Discount Calculation)

👉 10% discount

```
SELECT title, price,
       price - (price * 0.10) AS discounted_price
FROM Books;
```

◆ 6. Logical Condition

```
SELECT *
FROM Books
WHERE price > 500
AND author = 'XYZ';
```

◆ 7. Pattern Matching

```
SELECT *
FROM Books
WHERE title LIKE '%SQL%';
👉 Finds books containing "SQL"
```

◆ 8. String Functions

```
SELECT LOWER(title),
       CONCAT(title, ' by ', author) AS book_details
FROM Books;
```

◆ **9. Set Operator (UNION)**

SELECT name FROM Members

UNION

SELECT author FROM Books;

👉 Combines member names and author names

🎯 **Viva Points (VERY IMPORTANT)**

● **Arithmetic Operation**

price - (price * 0.10)

👉 Discount calculation

● **Pattern Matching**

LIKE '%SQL%'

👉 Contains SQL

● **String Functions**

- LOWER() → lowercase
 - CONCAT() → joins strings
-

● **UNION**

- Combines outputs
 - Removes duplicates
-

● **Foreign Key**

- Maintains relationship between tables

Library Management System with DCL & TCL

◆ 1. Create Tables

Members Table

```
CREATE TABLE Members (
  member_id INT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(50)
);
```

Books Table

👉 Add stock column for transaction handling

```
CREATE TABLE Books (
  book_id INT PRIMARY KEY,
  title VARCHAR(100),
  author VARCHAR(50),
  price INT,
  stock INT
);
```

Borrow Table

```
CREATE TABLE Borrow (
  borrow_id INT PRIMARY KEY,
  member_id INT,
  book_id INT,
  borrow_date DATE,
  FOREIGN KEY (member_id) REFERENCES Members(member_id),
  FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

◆ 2. Insert Sample Records

Members

```
INSERT INTO Members VALUES (1, 'Amit', 'amit@gmail.com');
INSERT INTO Members VALUES (2, 'Riya', 'riya@gmail.com');
```

Books

```
INSERT INTO Books VALUES (101, 'SQL Basics', 'XYZ', 500, 5);
INSERT INTO Books VALUES (102, 'Python Guide', 'ABC', 700, 3);
```

◆ 3. Create Roles

```
CREATE ROLE librarian;
CREATE ROLE assistant;
```

◆ 4. GRANT Permissions

Full Access to Librarian

```
GRANT ALL PRIVILEGES
ON Books
TO librarian;
```

SELECT Only to Assistant

```
GRANT SELECT
ON Books
TO assistant;
```

◆ 5. REVOKE Permission

```
REVOKE SELECT
ON Books
FROM assistant;
```

◆ 6. TCL Commands (Transaction Handling)

Start Transaction

```
START TRANSACTION;
```

Issue a Book (Insert into Borrow)

```
INSERT INTO Borrow
VALUES (1, 1, 101, SYSDATE);
```

Create SAVEPOINT

```
SAVEPOINT before_stock_update;
```

Update Book Stock

```
UPDATE Books
SET stock = stock - 1
WHERE book_id = 101;
```

Rollback if Stock Not Available

```
ROLLBACK TO before_stock_update;
```

👉 Used if stock becomes 0 or error occurs

Commit Transaction

```
COMMIT;
```

👉 Saves changes permanently

Banking System – Transaction Handling using TCL

◆ 1. Create Table

```
CREATE TABLE Accounts (
  acc_no INT PRIMARY KEY,
  cust_name VARCHAR(50),
  balance INT
);
```

◆ 2. Insert Sample Records

```
INSERT INTO Accounts VALUES (101, 'Amit', 50000);
INSERT INTO Accounts VALUES (102, 'Riya', 30000);
```

◆ 3. Display Initial Data

```
SELECT * FROM Accounts;
```

◆ 4. Start Transaction

```
START TRANSACTION;
```

◆ 5. Deduct Amount from Sender

👉 Amit sends 10000 to Riya

```
UPDATE Accounts
SET balance = balance - 10000
WHERE acc_no = 101;
```

◆ 6. Create SAVEPOINT

```
SAVEPOINT deduct_done;
```

◆ 7. Credit Amount to Receiver

```
UPDATE Accounts
SET balance = balance + 10000
WHERE acc_no = 102;
```

◆ 8. Simulate Error Condition

👉 Example: insufficient balance / system failure

```
ROLLBACK TO deduct_done;
```

👉 Reverts transaction to savepoint

◆ 9. Commit Transaction (if successful)

```
COMMIT;
```

👉 Saves all changes permanently

Complete Flow Explanation

Scenario:

- ₹10,000 transferred from Amit → Riya

Steps:

1. Transaction starts
2. Deduct money from Amit
3. Savepoint created
4. Add money to Riya
5. If error occurs → rollback
6. Else → commit

Viva Points (VERY IMPORTANT)

● START TRANSACTION

- Begins transaction block

● SAVEPOINT

- Creates checkpoint inside transaction

● ROLLBACK

- Undo changes after savepoint

● COMMIT

- Permanently saves changes

● Why TCL Important?

👉 Maintains:

- Data consistency
- Data integrity

MongoDB CRUD Operations

◆ 1. Create Database & Collection

use studentDB

```
db.createCollection("students")
```

◆ 2. CREATE Operation (Insert Documents)

👉 Using save() method

```
db.students.save({
  _id: 1,
  name: "Amit",
  age: 21,
  city: "Pune",
  marks: 85
})
```

```
db.students.save({
  _id: 2,
  name: "Riya",
  age: 22,
  city: "Mumbai",
  marks: 90
})
```

```
db.students.save({
  _id: 3,
  name: "Karan",
  age: 20,
  city: "Delhi",
  marks: 75
})
```

◆ 3. READ Operation

Display All Documents

```
db.students.find()
```

Find Specific Data

```
db.students.find({ city: "Pune" })
```

◆ 4. Logical Operators

AND Operator

```
db.students.find({
  $and: [
    { city: "Mumbai" },
    { marks: { $gt: 80 } }
  ]
})
```

OR Operator

```
db.students.find({
  $or: [
    { city: "Pune" },
    { marks: { $gt: 85 } }
  ]
})
```

◆ 5. UPDATE Operation

```
db.students.updateOne(
  { name: "Amit" },
  { $set: { marks: 95 } }
)
```

👉 Updates Amit's marks

◆ 6. DELETE Operation

```
db.students.deleteOne({
  name: "Karan"
})
```

👉 Deletes Karan's document

◆ 7. Display Final Records

```
db.students.find()
```

🎯 Viva Points (VERY IMPORTANT)

● save() Method

- Inserts document if not present
- Updates if _id already exists

Banking Database Queries (Set Operations & Tuple Variables)

◆ 1. Create Tables

Depositor Table

```
CREATE TABLE Depositor (
    acc_no INT,
    cname VARCHAR(50),
    branch_name VARCHAR(50)
);
```

Borrower Table

```
CREATE TABLE Borrower (
    loan_no INT,
    cname VARCHAR(50),
    branch_name VARCHAR(50),
    amount INT
);
```

Branch Table

```
CREATE TABLE Branch (
    branch_name VARCHAR(50),
    city VARCHAR(50),
    assets INT
);
```

◆ 2. Insert Sample Records

Depositor

```
INSERT INTO Depositor VALUES (101, 'Amit', 'Perryridge');
INSERT INTO Depositor VALUES (102, 'Riya', 'Brighton');
INSERT INTO Depositor VALUES (103, 'Karan', 'Perryridge');
INSERT INTO Depositor VALUES (104, 'Neha', 'Downtown');
INSERT INTO Depositor VALUES (105, 'Rahul', 'Brooklyn');
```

Borrower

```
INSERT INTO Borrower VALUES (201, 'Amit', 'Perryridge', 5000);
INSERT INTO Borrower VALUES (202, 'Riya', 'Brighton', 7000);
INSERT INTO Borrower VALUES (203, 'Arjun', 'Brooklyn', 9000);
INSERT INTO Borrower VALUES (204, 'Neha', 'Downtown', 6000);
INSERT INTO Borrower VALUES (205, 'Karan', 'Perryridge', 8000);
```

Branch

```
INSERT INTO Branch VALUES ('Perryridge', 'New York', 90000);
INSERT INTO Branch VALUES ('Brighton', 'Chicago', 70000);
INSERT INTO Branch VALUES ('Brooklyn', 'Brooklyn', 50000);
INSERT INTO Branch VALUES ('Downtown', 'Boston', 80000);
INSERT INTO Branch VALUES ('Central', 'Miami', 100000);
```

Queries

◆ 3. Customers having a loan, an account, or both

☞ UNION

```
SELECT cname FROM Depositor
```

```
UNION
```

```
SELECT cname FROM Borrower;
```

◆ 4. Customers having both loan and account

☞ INTERSECT

```
SELECT cname FROM Depositor
```

```
INTERSECT
```

```
SELECT cname FROM Borrower;
```

◆ 5. Customers having account but no loan

☞ MINUS / EXCEPT

```
SELECT cname FROM Depositor
```

```
MINUS
```

```
SELECT cname FROM Borrower;
```

☞ Use EXCEPT in MySQL/PostgreSQL if MINUS not supported

◆ 6. Customer names, loan numbers, amount using Tuple Variable

```
SELECT b.cname, b.loan_no, b.amount
```

```
FROM Borrower b;
```

☞ b is tuple variable (table alias)

◆ 7. Branches having greater assets than some branch in Brooklyn

```
SELECT branch_name
```

```
FROM Branch
```

```
WHERE assets > SOME (
```

```
SELECT assets
FROM Branch
WHERE city = 'Brooklyn'
);
```

◆ **8. Customers having loan in Perryridge branch (Alphabetical Order)**

```
SELECT cname
FROM Borrower
WHERE branch_name = 'Perryridge'
ORDER BY cname ASC;
```

🚩 **Viva Points (VERY IMPORTANT)**

● **UNION**

- Combines results
 - Removes duplicates
-

● **INTERSECT**

- Common values from both queries
-

● **MINUS**

- Values in first query but not second
-

● **Tuple Variable**

Borrower b

👉 b is alias for table

● **ORDER BY**

ORDER BY cname ASC

👉 Sorts alphabetically

● **SOME Operator**

- Compares with at least one value from subquery

 Company Database SQL Solution◆ **Create Tables**

```
CREATE TABLE dept(  
  deptno INT PRIMARY KEY,  
  deptname VARCHAR(50),  
  location VARCHAR(50)  
);  
  
CREATE TABLE emp(  
  eno INT PRIMARY KEY,  
  ename VARCHAR(50),  
  job VARCHAR(50),  
  hiredate DATE,  
  salary INT,  
  commission INT,  
  deptno INT,  
  FOREIGN KEY(deptno) REFERENCES dept(deptno)  
);
```

◆ **Insert Records**

```
INSERT INTO dept VALUES (10,'Dev','Pune');  
INSERT INTO dept VALUES (20,'HR','Mumbai');  
  
INSERT INTO emp VALUES (1,'Ishan','Salesman','2023-01-10',50000,2000,10);  
INSERT INTO emp VALUES (2,'Amit','Manager','2022-05-15',70000,3000,10);  
INSERT INTO emp VALUES (3,'Irfan','Salesman','2021-03-12',45000,1500,20);  
INSERT INTO emp VALUES (4,'Neha','Clerk','2020-07-20',30000,1000,20);
```

◆ **1. Maximum salary paid to salesman**

```
SELECT MAX(salary)  
FROM emp  
WHERE job='Salesman';
```

◆ **2. Employee names starting with 'I'**

```
SELECT ename  
FROM emp  
WHERE ename LIKE 'I%';
```

◆ **3. Employee details in descending order of salary**

```
SELECT *  
FROM emp  
ORDER BY salary DESC;
```

◆ **4. Average & minimum salary hiredate-wise for dept 10**

```
SELECT hiredate,  
  AVG(salary),  
  MIN(salary)  
FROM emp  
WHERE deptno=10  
GROUP BY hiredate;
```

◆ **5. Employee name and department**

```
SELECT e.ename, d.deptname  
FROM emp e  
JOIN dept d  
ON e.deptno=d.deptno;
```

◆ **6. Total salary paid to each department**

```
SELECT deptno,  
  SUM(salary)  
FROM emp  
GROUP BY deptno;
```

◆ **7. Employee details working in 'Dev' department**

```
SELECT e.*  
FROM emp e  
JOIN dept d  
ON e.deptno=d.deptno  
WHERE d.deptname='Dev';
```